



CIN0136

Desenvolvimento de Software

Desenvolvimento de Software

Prof. Vinicius Cardoso Garcia

vcg@cin.ufpe.br :: @vinicius3w :: assertlab.com

<http://viniciusgarcia.me>



**Centro de
Informática**
UFPE



UNIVERSIDADE
FEDERAL
DE PERNAMBUCO

Licença do material

Este Trabalho foi licenciado com uma Licença



**Atribuição-NãoComercial-
Compartilha Igual 4.0 Internacional
(CC BY-NC-SA 4.0)**

Mais informações visite

<https://creativecommons.org/licenses/by-nc-sa/4.0/deed.pt> BR



Engenharia de Software em Dimensões

É um material abrangente, didático e atualizado que apresenta a Engenharia de Software como uma disciplina multidimensional. Cada capítulo foca em uma dimensão prática, conectando teoria e realidade profissional com clareza, profundidade e propósito.

Prof. Vinicius Cardoso Garcia

<https://bit.ly/esdbook>

Engenharia de Software em Dimensões

Da base conceitual à prática profissional



Vinicius Cardoso Garcia



Ponto de Commit

O podcast figital da disciplina (IF977) Engenharia de Software do curso de bacharelado em Sistemas de Informação do Centro de Informática (CIn) da Universidade Federal de Pernambuco (UFPE).

<https://go.uaite.xyz/ponto-de-commit>

Ponto de Commit



O podcast figital da disciplina de Engenharia de Software



Engenharia de Software Moderna

Engenharia de Software Moderna é um livro destinado a alunos de cursos de graduação em Computação. Ele pode ser lido também por profissionais que buscam um conhecimento horizontal da área.

Prof. Marco Tulio Valente

<https://engsoftmoderna.info>

@engsoftmoderna

ENGENHARIA — DE — SOFTWARE MODERNA

Princípios e práticas para
desenvolvimento de software
com produtividade



MARCO TULIO VALENTE





Debugging e Robustez

Desenvolva habilidades de debugging, construa aplicações mais resistentes e prepare demonstrações que impressionam stakeholders



Quem já perdeu mais de uma hora nisso?

"Um bug que, quando você finalmente encontrou, era uma linha de código, mas você não sabia onde procurar."

Quase todo mundo já viveu isso. O problema não é o bug, é a **busca**. Hoje vamos mudar isso.



O fluxo que parece natural (mas é ineficiente)

O que a maioria faz



Bug surge

Console.log

Confusão

Chute

Bug some

O que falta em cada etapa

→ Sem reprodução consistente

Se você não consegue fazer o bug acontecer de novo, como vai saber se consertou?

→ Sem hipótese

Um log sem hipótese é ruído. Você não sabe o que está procurando.

→ Sem garantia

O bug "sumiu", mas vai voltar. Não há teste que prove que está resolvido.



Debugging eficiente = investigação com hipótese



 **A diferença fundamental:** você não adiciona log porque está perdido — você adiciona log para **testar uma hipótese específica**. O log é um instrumento de medição, não um tiro no escuro.

O Desafio: Bugs Ocultos e Demonstrações Frustradas

O Custo Invisível

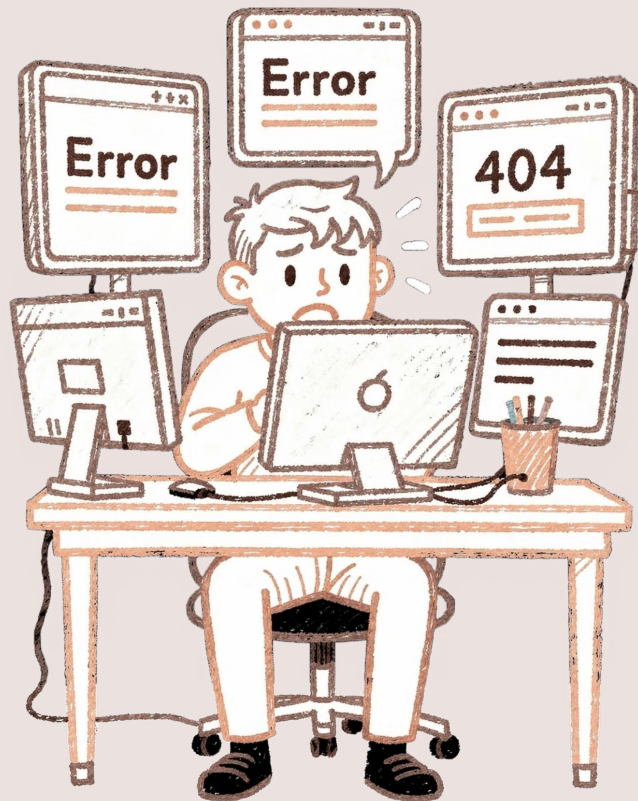
Bugs inesperados durante demonstrações cruciais destroem a credibilidade e geram desconfiança imediata no stakeholder.

Percepção de Qualidade

Uma falha ao vivo compromete semanas de trabalho e pode redefinir negativamente a percepção do produto.

Abordagem Proativa

A solução está em uma estratégia sistemática de debugging — prevenir antes de remediar.



Breakpoints: o superpoder que você provavelmente não está usando

O que é um breakpoint

Pausa a execução do programa em uma linha específica. Você entra no tempo congelado do seu código.

O que você pode inspecionar

Tudo: variáveis locais, pilha de chamadas, estado dos objetos, exatamente no momento do problema.

Compatibilidade

Funciona para **Node.js**, **Express** e **React**, sem instalar nada além do VS Code.

Como ativar em 3 passos

01

Marque a linha

Clique na margem esquerda ao lado do número da linha → aparece um ponto vermelho.

02

Inicie o debugger

Pressione F5 → selecione "**Node.js**" no menu que aparecer.

03

Dispare a requisição

Faça o request via browser, Postman ou teste, a execução pausa na linha marcada.



As três janelas que importam no debugger



VARIABLES

Variáveis locais, de closure e globais no exato momento da pausa. Expanda objetos para ver cada propriedade.



CALL STACK

Sequência de chamadas que chegou até aqui. **Leia de baixo para cima**, a origem está nas camadas inferiores.



WATCH

Expressões que você quer monitorar continuamente: `req.body`, `user.id`, qualquer expressão JavaScript válida.

Tecla

F10

F11

F5

Ação

Step Over

Step Into

Continue

Quando usar

Executa a linha atual e vai para a próxima, sem entrar em funções

Entra na função que está sendo chamada na linha atual

Retoma até o próximo breakpoint ou fim da execução



Corrigir + garantir que não volta

✗ Antes — 🐛 quebrado

```
const user = await db.findUser(req.params.id);  
// explode se user for null  
res.json({ name: user.name });
```

✓ Depois — correto

```
const user = await db.findUser(req.params.id);  
if (!user) {  
  return res.status(404).json({  
    error: 'Usuário não encontrado'  
  });  
}  
res.json({ name: user.name });
```

🔧 Teste que garante que não volta

```
test('GET /users/:id retorna 404  
  para ID inexistente', async () => {  
  const res = await request(app)  
    .get('/users/9999');  
  
  expect(res.status).toBe(404);  
});
```

A correção resolve o presente. O teste protege o futuro.
Sem o teste, o bug pode voltar silenciosamente em
qualquer refactor.





Visual Studio Code: Seu Aliado no Debugging

- **Suporte nativo** para JavaScript e React sem configuração extra.
- Ferramentas poderosas integradas para identificar e corrigir problemas rapidamente.
- Integração perfeita com o ecossistema de desenvolvimento moderno: npm, Git, extensões.



Breakpoints Estratégicos no VS Code

Pausar Execução

Executar e aguardar a pausa.

Inspecionar Variáveis

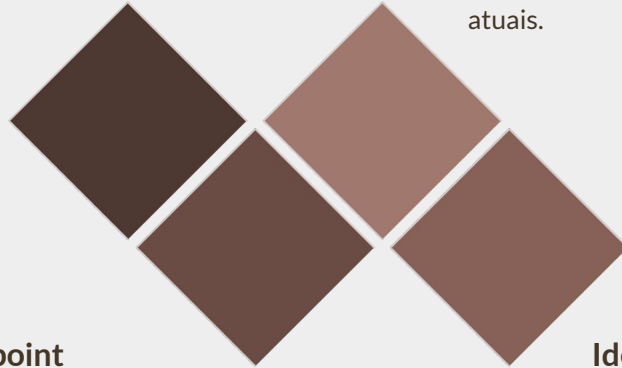
Ver valores e estruturas atuais.

Definir Breakpoint

Marcar linha onde quer pausar.

Identificar Causa

Determinar origem do erro.



Breakpoints permitem pausar a execução exatamente onde o problema acontece, inspecionar o estado completo da aplicação e identificar a causa raiz com precisão cirúrgica, eliminando suposições do processo de debugging.



Console.log: O Clássico Reinventado



Uso Estratégico

Posicione logs em pontos-chave do fluxo para rastrear valores, estados e mudanças ao longo da execução.



Combinação Poderosa

Use `console.log` junto com breakpoints para uma análise completa, logs revelam o "o quê", breakpoints revelam o "por quê".



Logs Informativos

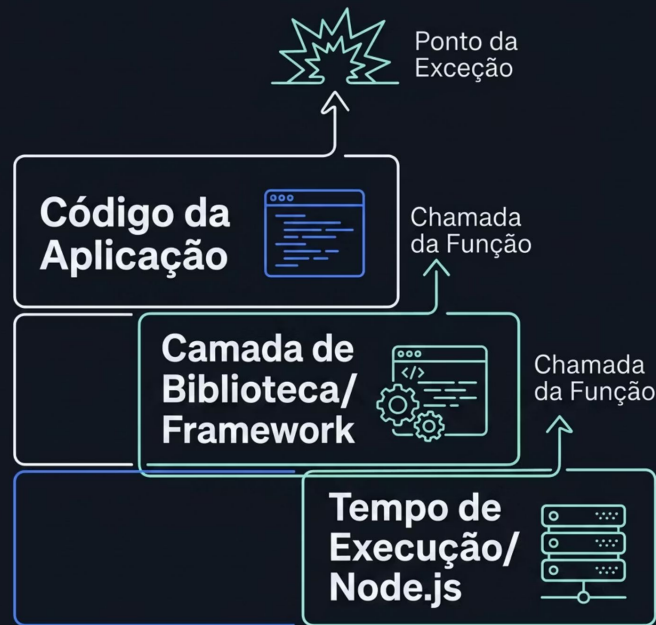
Prefixe seus logs com contexto:
`console.log(' [UserService] ', data)` para facilitar a leitura em produção.



Desvendando Stack Traces

Um stack trace é o rastro completo de chamadas de função que levaram a um erro. Ler um stack trace de baixo para cima revela a sequência exata de eventos, desde a origem até o ponto de falha.

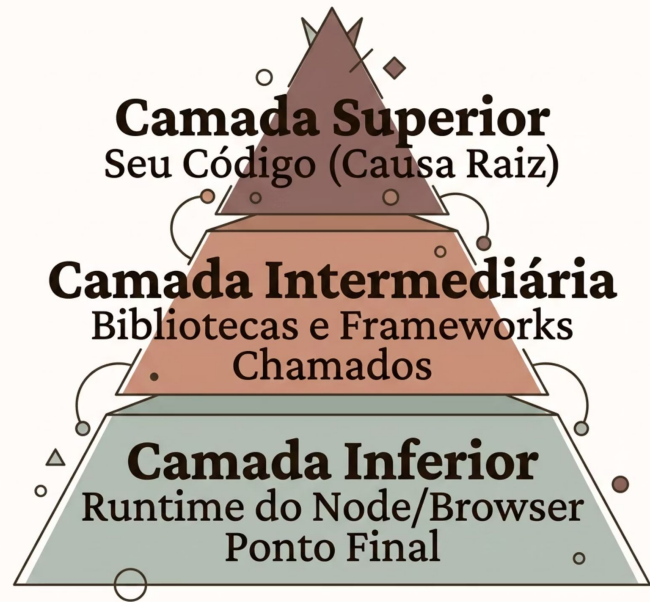
i Dica: sempre comece lendo a primeira linha do erro, ela contém o tipo e a mensagem mais relevante.



Desvendando Stack Traces

Um stack trace é o rastro completo de chamadas de função que levaram a um erro. Ler um stack trace de baixo para cima revela a sequência exata de eventos, desde a origem até o ponto de falha.

i Dica: sempre comece lendo a primeira linha do erro, ela contém o tipo e a mensagem mais relevante.





Construindo Robustez com Tratamento de Erros

Prevenir falhas catastróficas e garantir que a aplicação responda de forma controlada e elegante a qualquer situação adversa.



A Importância do Tratamento de Erros



Estabilidade Garantida

Previne falhas catastróficas que derrubam a aplicação inteira por um único erro não tratado.



Experiência Superior

Usuários recebem mensagens claras e contextuais em vez de telas brancas ou mensagens de erro técnicas.



Controle Total

Responda a erros de forma controlada (com fallbacks, retries e logs) sem comprometer o fluxo principal.



try/catch estratégico: não é para envolver tudo

✅ Use para: razões fora do seu controle

→ Chamadas ao banco de dados

Conexão pode falhar, timeout pode ocorrer.

→ Requests externos (APIs de terceiros)

Serviço pode estar indisponível ou retornar erro.

→ Leitura/escrita de arquivos

Arquivo pode não existir ou falta de permissão.

→ JSON.parse de dados externos

O JSON pode estar malformatado, nunca confie em dados externos.

❌ Não use para: erros de lógica

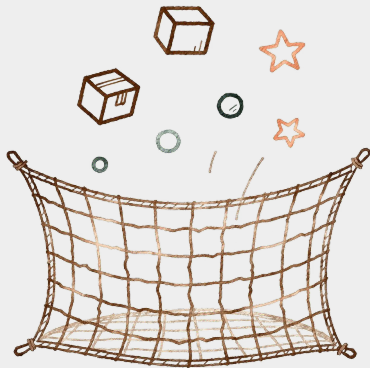
Erros de lógica que você deveria estar **corrigindo** não devem ser silenciados por try/catch. Envolver demais mascara bugs reais e dificulta o debugging.



Um try/catch muito amplo transforma **bugs visíveis** em **falhas silenciosas**. O usuário vê "Something went wrong" e você não sabe o que aconteceu.



Try/Catch: O Guardião do Código



O bloco `try/catch` é a primeira linha de defesa contra exceções inesperadas. Ele permite capturar erros de forma elegante, implementar lógica de fallback e recuperação, e evitar que um único ponto de falha derrube toda a aplicação.



Boas práticas: sempre logar o erro capturado e nunca usar `catch` vazio, trate ou relance com contexto.

try { }

Código que pode gerar exceção é executado aqui com segurança.

catch (e) { }

Captura a exceção e executa a lógica de recuperação ou fallback.

finally { }

Executado sempre, ideal para limpeza de recursos e encerramento.



O problema do try/catch genérico

❌ Genérico demais: mascara bugs

```
try {
  const users = await getUsers();
  // erro de lógica aqui fica escondido
  const filtered = users.filter(u => u.active);
  res.json(filtered);
} catch (err) {
  res.status(500).json({
    error: 'Something went wrong'
  });
}
```

✅ Específico: captura o que pode falhar

```
const users = await getUsers()
  .catch(err => {
    console.error('[DB] Falha ao buscar usuários:',
                  err.message);
    res.status(503).json({
      error: 'Database unavailable'
    });
    return null;
  });

if (!users) return;

// erro de lógica fica visível aqui
const filtered = users.filter(u => u.active);
res.json(filtered);
```

🕒 No segundo exemplo, se houver um `TypeError` no `.filter()`, ele aparece como uma exceção não tratada visível no log, fácil de encontrar.



Respostas HTTP Adequadas: Comunicação Clara com o Backend



2xx — Sucesso

200 OK, 201 Created: operação realizada com êxito. Retorne os dados esperados no body.



4xx — Erro do Cliente

400 Bad Request, 404 Not Found, 401 Unauthorized: o problema está nos dados enviados. Retorne mensagens descritivas.



5xx — Erro do Servidor

500 Internal Server Error: nunca exponha stack traces em produção. Log interno + mensagem genérica para o cliente.



<api design> O status HTTP que você retorna informa quem tem que resolver o problema

4xx — o cliente fez algo errado

Status	Quando usar
400	Dados inválidos no corpo da requisição
401	Token ausente ou inválido
403	Autenticado, mas sem permissão
404	Recurso não encontrado
422	Dados válidos no formato, inválidos no negócio

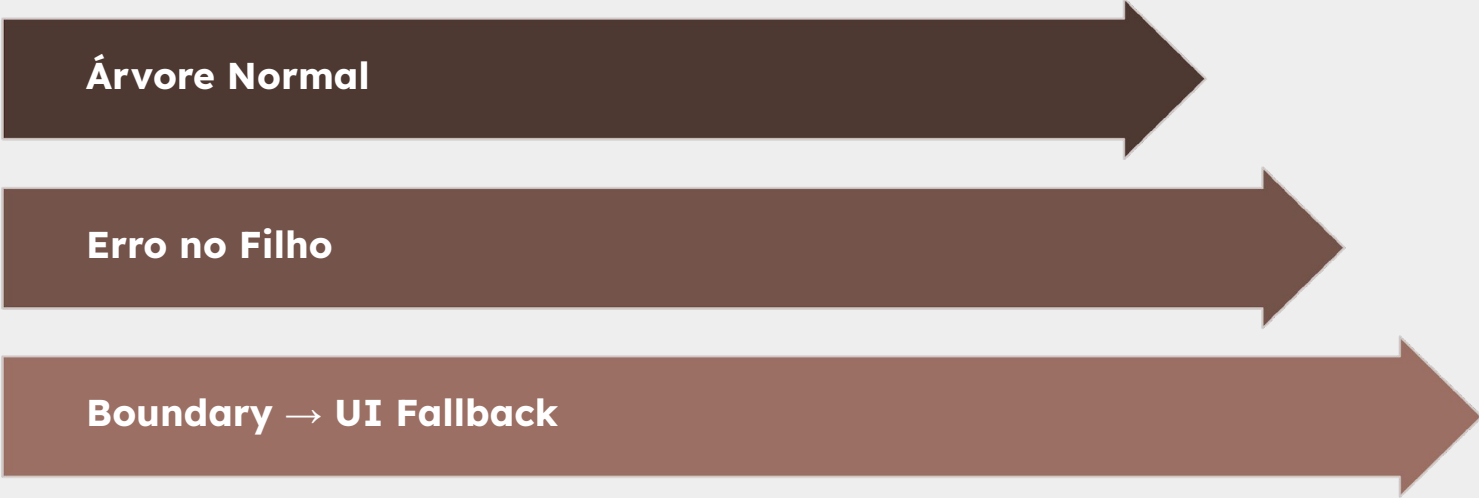
5xx — o servidor falhou

Status	Quando usar
500	Erro inesperado no servidor
503	Serviço dependente indisponível (banco fora do ar)

⚠ **Retornar 500 para tudo é um problema de debugging**, não só de UX. O frontend não consegue diferenciar "você errou" de "nós erramos".



Error Boundaries em React: Protegendo a UI



Árvore Normal

Erro no Filho

Boundary → UI Fallback

Error Boundaries isolam falhas de componentes individuais, impedindo que um único erro se propague e quebre a aplicação inteira. A UI permanece funcional e o usuário vê uma mensagem amigável em vez de uma tela em branco.



<react> Error Boundary, evite a tela branca

Por que importa

✗ Sem Error Boundary

Erro em um componente → tela branca para o usuário inteiro.

✓ Com Error Boundary

Erro contido → fallback controlado → usuário vê mensagem útil.

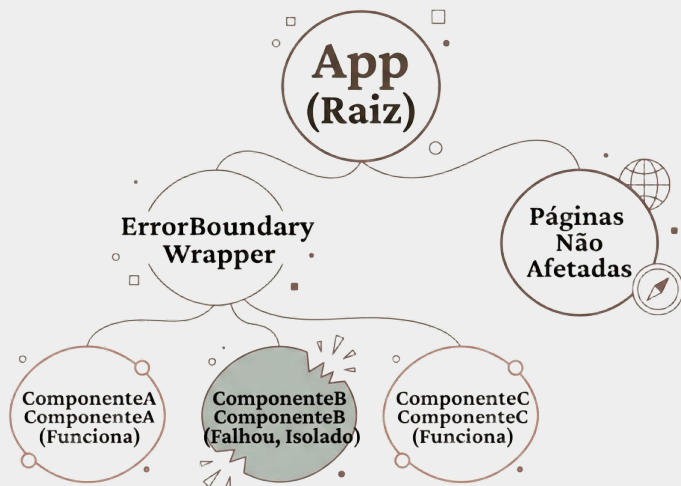
Implementação mínima

```
class ErrorBoundary extends Component {
  state = { hasError: false };

  static getDerivedStateFromError() {
    return { hasError: true };
  }

  render() {
    if (this.state.hasError) {
      return (
        <div>
          Algo deu errado.
          <button onClick={...}>
            Tentar novamente
          </button>
        </div>
      );
    }
    return this.props.children;
  }
}

// Uso:
<ErrorBoundary>
  <ComponenteQuePodeFalhar />
</ErrorBoundary>
```



<logging> console.log ainda tem lugar — mas com contexto

❌ Inútil — sem contexto

```
console.log(data);
```

O que é data? De qual função veio? Em qual fluxo? Impossível saber sem abrir o arquivo.

✅ Útil — com localização e conteúdo

```
console.log(
  '[userService.getById] userId recebido:',
  userId
);
console.log(
  '[userService.getById] resultado do banco:',
  JSON.stringify(user, null, 2)
);
```

✅ Para erros — com contexto estruturado

```
console.error(
  '[routes/users] Falha no GET /users/:id',
  {
    userId: req.params.id,
    error: err.message
  }
);
```

Regra prática: se você vai adicionar um log, seja explícito sobre **onde está** (módulo/função) e **o que está imprimindo** (nome da variável). Logs anônimos são inúteis às 3h da manhã.



Exercício rápido, leia este stack trace

Workout Plan

```
Error: ENOENT: no such file or directory,  
    open '/app/config/database.json'  
    at Object.openSync (node:fs:596:3)  
    at Object.readFileSync (node:fs:464:35)  
    at loadConfig (/app/utils/config.js:8:18) ← seu código  
    at Object.<anonymous> (/app/server.js:3:1) ← seu código
```

1. Qual é o erro?

Descreva com suas palavras o que está falhando.

2. Onde está a causa?

Qual arquivo do **seu** código é a origem?

3. O que está acontecendo?

Qual hipótese você formularia antes de abrir o editor?

  **3 minutos:** responda individualmente, depois corrigimos coletivamente.



Exercício em equipe: debugging com hipótese

1

Acesse o seu repositório

Se pertinente, clone e instale as dependências.

2

Se reúna em equipe

Preferencialmente com alguém que atuou em **outra feature**, perspectiva externa ajuda.

3

Encontre 1 bug por membro

Use **breakpoints** e **leitura de stack trace**, não tentativa e erro. Anote a hipótese antes de abrir o debugger. Registre tudo.

4

Envie todos os registros

Registre o trabalho de todos em um documento único, identificando a equipe, o contexto, o link pro repositório do projeto e os seus membros.

Os bugs usualmente podem envolver:

TypeError por retorno `null` não tratado

JSON.parse sem try/catch

Promise não tratada

Status HTTP semanticamente errado

TypeError em cadeia de propriedades



A demo de não pode falhar nesses pontos

● Fluxos críticos — devem funcionar sem falhas

→ Login / autenticação

Se aplicável ao seu sistema, sem exceções.

→ Funcionalidade principal do MVP

O que o stakeholder veio ver. Tem que funcionar.

→ Criação de dados

Formulário → banco → confirmação visível para o usuário.

→ Listagem e visualização

Os dados criados precisam aparecer.

● Tratamento mínimo obrigatório

Loading state

Quando o banco demora, o usuário precisa saber que está carregando.

Validação visível

Formulários devem mostrar erros legíveis, não `console.error`.

Sem tela branca

Em nenhum cenário esperado. Use Error Boundary.

Dados reais

Banco com dados de demonstração, não "teste", "aaa", "123".



tempo de dev antes da review, use bem

1

Prioridade 3 — Error handling visível

Mensagens de erro amigáveis nos fluxos que podem falhar. O stakeholder nunca deve ver um stack trace.

2

Prioridade 2 — Dados de demonstração

Reserve **30 minutos** para popular o banco com dados que façam sentido para o stakeholder. Dados ruins quebram a narrativa.

3

Prioridade 1 — Estabilidade dos fluxos críticos

Nenhuma feature nova. Zero. Só garantir que o que existe funciona do início ao fim.

- ✓ O stakeholder avalia o que funciona, não o que foi construído. Uma feature que funciona vale mais do que três que quebram na demo.



Cenários de Teste: Cobrindo Todas as Bases

✓ Happy Path

Fluxo ideal de ponta a ponta, sem erros ou dados faltando.

⚠ Edge Cases

Inputs inválidos, campos vazios, dados extremos ou fora do esperado.

✗ Error Scenarios

Erros de API, timeout de rede e respostas inesperadas do backend.

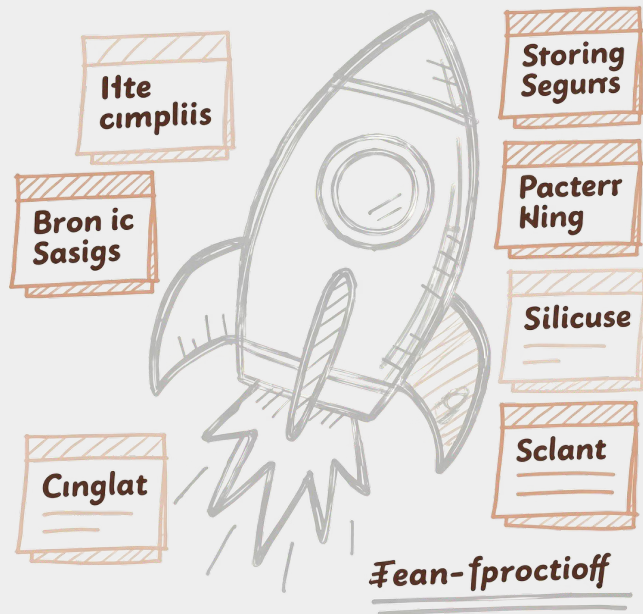
Por que testar todos os cenários?

Simular o uso real da aplicação (incluindo os fluxos de erro) garante que nenhuma surpresa apareça durante a demo.

Cenários de erro controlados, quando bem tratados, impressionam positivamente o stakeholder.



Feedback e Iteração: O Ciclo Contínuo de Melhoria



Coletar Feedback

Perguntas e observações dos stakeholders após a demo são ouro puro.



Refinar

Debugging e tratamento de erros evoluem com cada iteração do produto.

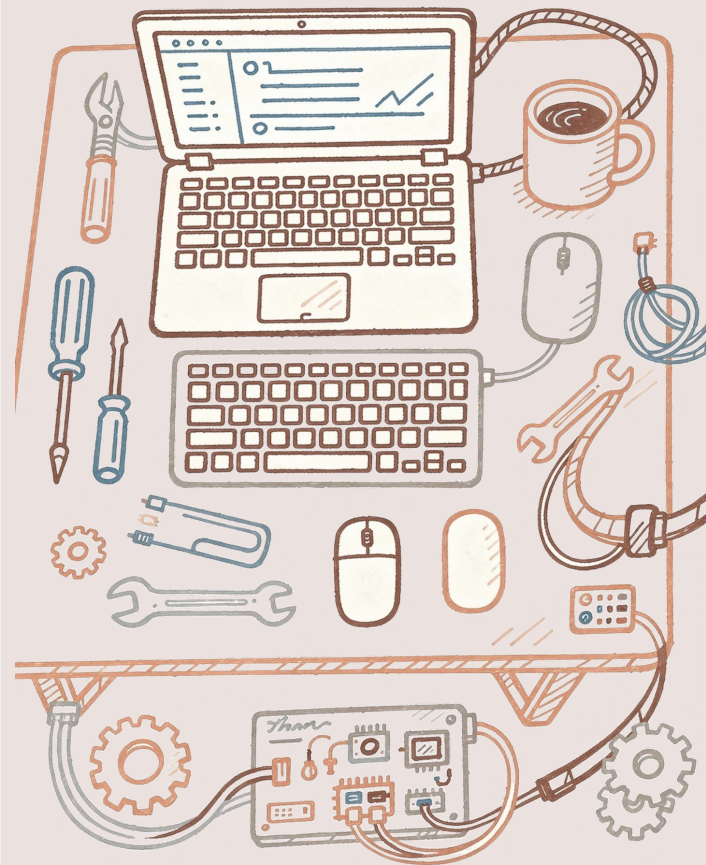
Demonstrar

Cada sprint review é uma oportunidade de mostrar evolução e maturidade técnica.



Ferramentas Avançadas e Boas Práticas

Indo além do básico: configurações avançadas e recursos do ecossistema para elevar a qualidade do debugging em projetos React.



Debugging de Apps React com Create React App

Configuração do launch.json

O VS Code permite configurar o launch.json para depurar aplicações React diretamente no editor, com suporte a source maps e breakpoints no código JSX original, sem precisar do DevTools do browser.

- ❏ Adicione a extensão **Debugger for Chrome** ao VS Code para a melhor experiência de debugging com `create-react-app`

1. launch.json

Configure `type` :
`"chrome"` e aponte para
`localhost:3000`.

2. Source Maps

Habilitados por padrão no CRA, breakpoints mapeiam o JSX original.

3. React DevTools

Inspecione componentes, props e state em tempo real no browser.



Conclusão: Desenvolvedores Confiantes, Demonstrações Impecáveis

Debugging Eficaz

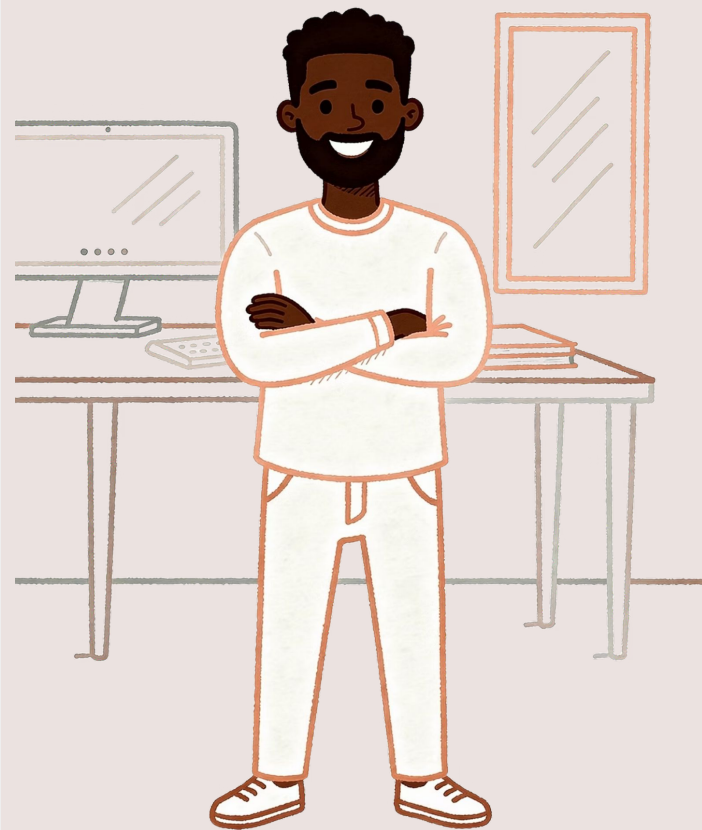
Breakpoints, console.log estratégico e stack traces são a base para encontrar qualquer bug com precisão.

Código Robusto

Try/catch, Error Boundaries e respostas HTTP adequadas protegem a aplicação e o usuário.

Demo Perfeita

Planejamento, ambiente estável e narrativa clara transformam código em valor percebido pelo stakeholder.



Resumo desta aula



Debugging = hipótese → teste → confirmar

Não tentativa e erro. Cada log é um instrumento de medição, não um tiro no escuro.



Breakpoints revelam o estado real do programa

Use antes do console.log. Stack trace: pare na primeira linha que é **seu** código.



try/catch específico e status HTTP semântico

Só ao redor do que pode falhar externamente. 4xx = cliente errou, 5xx = servidor falhou.



Na quinta: o checklist de robustez é o roteiro

Error Boundary para React, dados reais no banco, fluxos críticos funcionando. Sem features novas.



Bons bugs para vocês

Semana 11 · Terça-feira · Sprint 3

July
17

Próximo encontro

Quinta-feira — Sprint Review. 2h de debugging e desenvolvimento + preparação de apresentação para o stakeholder.



Antes de quinta

Percorra o checklist de robustez. Popule o banco. Teste os fluxos críticos do início ao fim.



Dúvidas

Canal do Discord da turma ou abertura de issue no repositório de exercícios.



Referências e recursos para se aprofundar



VS Code Debugger

Documentação oficial:

code.visualstudio.com/docs/editor/debugging: guia completo para Node.js e React, incluindo launch.json e configurações avançadas.



Node.js Error Handling

Guia oficial do Node.js sobre tratamento de erros assíncronos, domains e uncaughtException, base para APIs robustas.



React Error Boundaries

Documentação oficial do React:

react.dev/reference/react/Component#catching-rendering-errors, exemplos e boas práticas de Error Boundary.



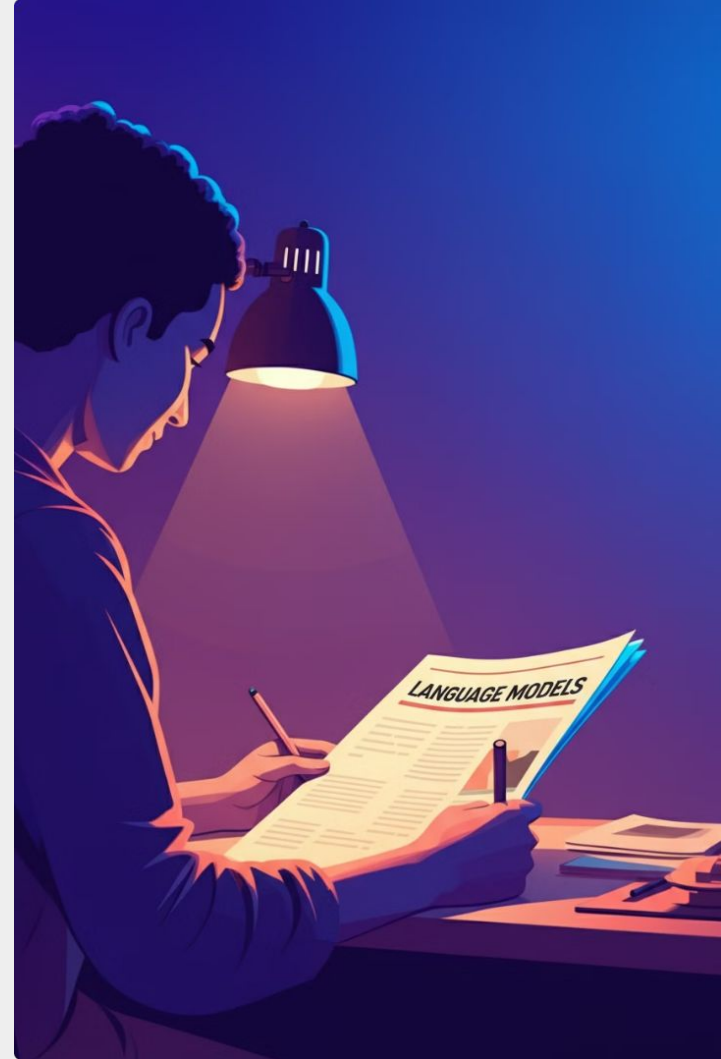
HTTP Status Codes

RFC 9110 ou httpstatuses.com: referência rápida para escolher o status correto em cada cenário da sua API.



Próximos passos

- Eng. Software em Dimensões
 - Cap. 14 (seções 14.1–14.8):
Arquitetura, C4 Model, qualidade arquitetural
- Eng. Software Moderna
 - "Cap. 5 — Princípios de Projeto Coesão, acoplamento, SOLID
 - Cap. 7 — Arquitetura Camadas, MVC, trade-offs"



contacts

vcg@cin.ufpe.br
[@vinicius3w](https://www.instagram.com/vinicius3w)
viniciusgarcia.me

academy

go.uaite.xyz/vcg-es
go.uaite.xyz/vcg-microservices
go.uaite.xyz/vcg-devops
go.uaite.xyz/vcg-esaia
go.uaite.xyz/vcg-deso

Desenvolvimento de Software (CIN0136)

Debugging e Robustez